AquaSentinel: Agentic AI for Dynamic Urban Waterlogging Response

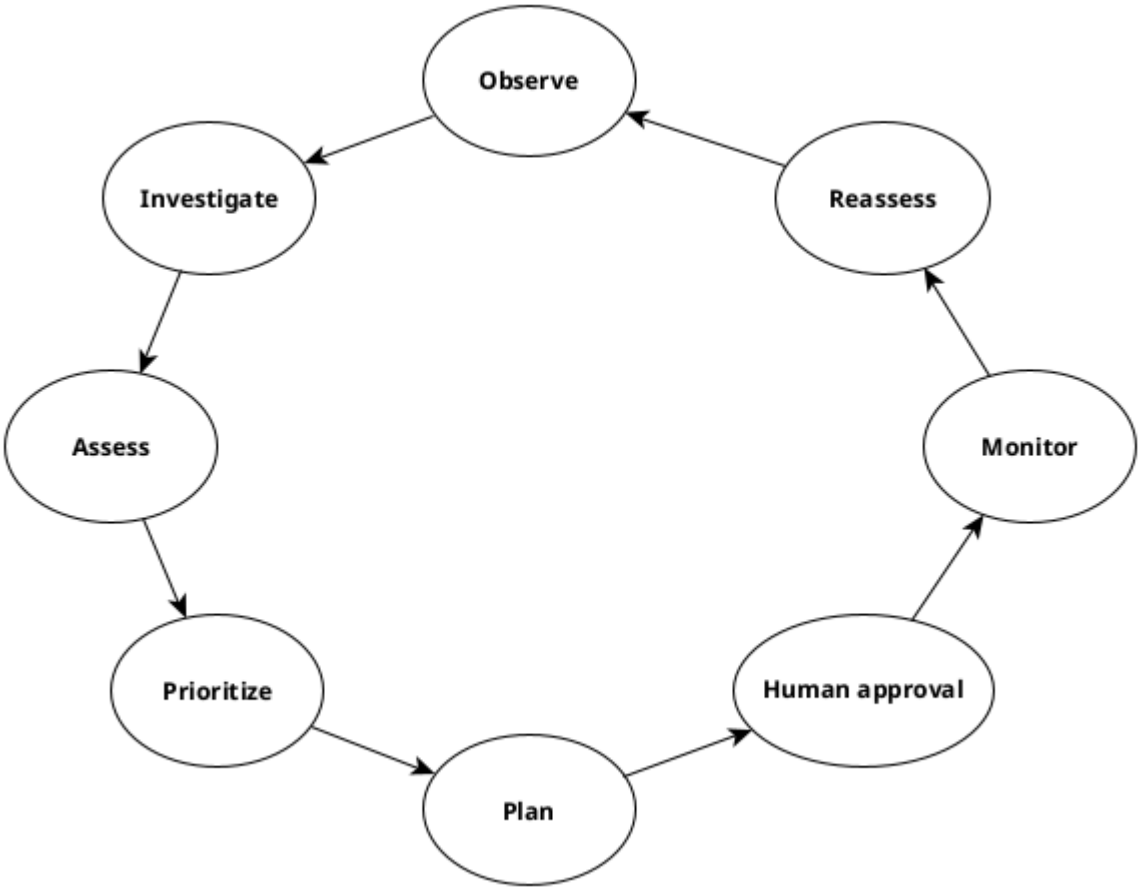
The project proposes an **Agentic AI-based decision-support system** for managing urban waterlogging during heavy rainfall.

Instead of trying to replace existing flood-forecasting systems, the system focuses on the **response and resource-allocation problem**.

The system receives information such as:

- Rainfall and weather forecasts
- Water-level changes
- Drainage conditions
- Geographic and elevation data
- Historical waterlogging hotspots
- Citizen/field reports
- Population and critical infrastructure
- Available pumps and response teams

An AI agent continuously analyzes this information and:



For example, if four areas are waterlogged but only three pumps are available, the agent determines which areas should receive priority based on factors such as **severity, rate of deterioration, population impact, critical infrastructure, and resource availability**.



The agent does not directly control emergency infrastructure. Instead, it generates an **explainable response recommendation** for a municipal officer, who can approve or reject it.

Example response plan

INCIDENT: Urban Waterlogging — Event #024
Generated: 10:32 AM
Plan Priority: High

Immediate actions

1. **Zone C — Priority 1**
    - Deploy Pump P1.
    - Dispatch Team T1.
    - Monitor water level every 10 minutes.
    - Flag railway-station access road as critical.
    - Recommend temporary traffic diversion.
2. **Zone B — Priority 2**
    - Deploy Pump P2.

    - Monitor hospital access route.
    - Keep emergency vehicle access clear.
    - Escalate if water level exceeds 0.8 m.
3. **Zone D — Priority 3**
    - Deploy Pump P3.
    - Continue monitoring water-level trends.
    - Prepare an additional team if water level rises rapidly.
4. **Zone A — Priority 4**
    - No immediate resource deployment.
    - Continue monitoring.

PEAS Representation

Component	Project-specific definition
P — Performance Measure	Accurate identification and prioritization of critical waterlogging zones; effective allocation of limited pumps/response teams; fast response planning; adaptability to changing conditions; minimizing affected population and critical infrastructure; quality and explainability of recommendations; avoiding unnecessary resource reallocations.
E — Environment	An urban area experiencing rainfall and waterlogging. The environment includes roads, drainage systems, low-lying areas, residential zones, hospitals, schools, population, weather conditions, water levels, historical hotspots, citizen reports, and available emergency resources.
A — Actuators	Since this is a decision-support system, the agent's actuators are primarily **digital**: generate priority rankings, recommend pump/team allocation, generate response plans, issue alerts/recommendations, request human approval, update/revise plans, and notify relevant departments. It does **not directly control physical infrastructure**.
S — Sensors	Weather/rainfall data, water-level sensors or simulated readings, flood forecasts, GIS/geographical data, drainage information, historical incident data, citizen/field reports, infrastructure databases, population data, and resource availability/status.

Tech stack

Layer	Technology	Purpose
Frontend	**React.js**	Municipal officer dashboard and visualization
Backend	**Java + Spring Boot**	REST APIs, authentication, incident management, resource management
Agentic AI	**Spring AI or LangChain4j**	Agent orchestration, tool calling, LLM integration
LLM	**Gemini / OpenAI API / local LLM**	Reasoning, tool selection, response-plan generation and explanation
ML	**Python + Scikit-learn**	Waterlogging/risk prediction
ML API	**FastAPI**	Expose Python ML models to Java backend
Database	**PostgreSQL**	Users, incidents, zones, resources, plans, historical data
Geospatial	**PostGIS**	Zones, roads, infrastructure, distances and spatial queries
Optimization	**Python OR-Tools**	Resource allocation and response-plan optimization
Real-time updates	**WebSockets**	Updating dashboard when conditions change
Caching/state	**Redis**	Temporary incident state, agent state and frequently accessed data
Maps	**Leaflet + OpenStreetMap**	Interactive city/waterlogging map
Deployment	**Docker**	Package Java, Python and database services
Version control	**Git + GitHub**	Collaboration and source control

Research Question

Can an agentic AI system dynamically coordinate urban-waterlogging response under changing conditions and limited resources more effectively than a conventional fixed decision pipeline?